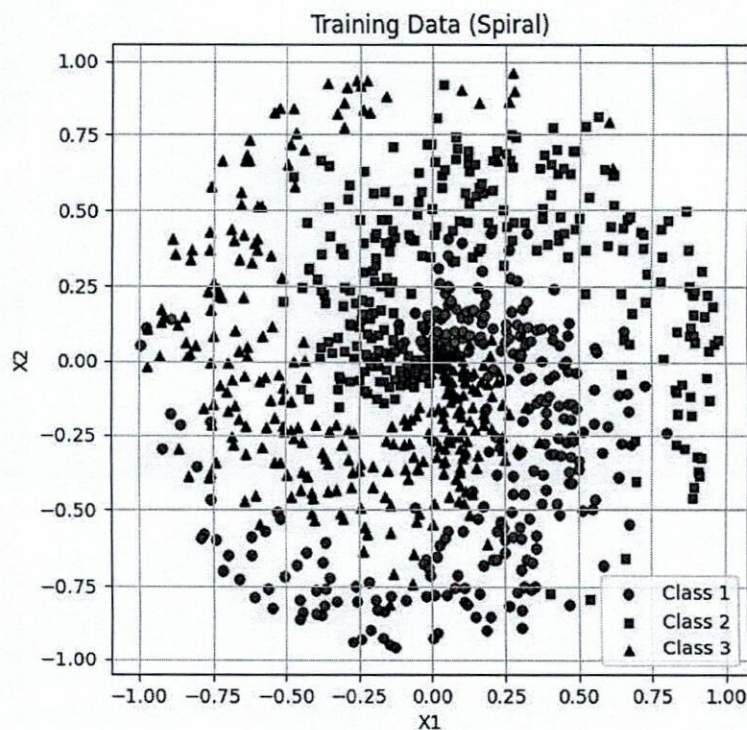


TRƯỜNG ĐH SƯ PHẠM KỸ THUẬT TP HCM KHOA CƠ KHÍ CHẾ TẠO MÁY		ĐỀ THI CUỐI HỌC KÌ I NĂM HỌC 2025-2026 Môn: Trí tuệ nhân tạo (Artificial Intelligence) Mã môn học: ARIN337629	
Chữ ký giám thị 1	Chữ ký giám thị 2	Đề số/Mã đề:	Đề thi có 11 trang.
Điểm và chữ ký		Thời gian: 90 phút.	
		Ngày thi: 29 / 10 / 2025	
		Được phép sử dụng tài liệu.	
		SV làm bài trực tiếp trên đề thi và nộp lại đề	
CB chấm thi thứ nhất	CB chấm thi thứ hai	Họ và tên:	
		Mã số SV:	
		Số TT: Phòng thi:	

Bài 1: Khởi tạo dữ liệu điểm (2 điểm)

Tạo một lớp SpiralData cho phép khởi tạo dữ liệu đám mây điểm ngẫu nhiên, với số lượng lớp (class) và số lượng điểm trong mỗi lớp (class) có thể được định nghĩa bởi người dùng. Ví dụ với 3 đám mây điểm (3 lớp), 300 điểm/lớp như hình dưới:



Hình 1: Đám mây điểm gồm 3 lớp (300 điểm/lớp) được khởi tạo bởi Class SpiralData


```

class SpiralData:
    def __init__(self, n_points, n_classes, n_dimensions):

        self.N = n_points # Số lượng điểm ở mỗi class
        self.D = n_dimensions # Chiều không gian
        self.K = n_classes # Số lượng lớp (class)

        # Khởi tạo ma trận P chứa dữ liệu điểm và vec-tơ nhãn dán L
        self.P = ... # câu 1.1
        self.L = ... # câu 1.1

        for j in range(self.K):
            ix = range(self.N*j, self.N*(j+1))
            r = ... # câu 1.2
            theta = ... # câu 1.2
            self.P[ix] = np.c_[r*np.sin(theta), r*np.cos(theta)]
            self.L[ix] = j

```

Câu 1.1 (1 điểm).

Xác định các câu lệnh cho phép khởi tạo ma trận chứa dữ liệu điểm self.P và vec-tơ nhãn dán cho các lớp self.L (dùng hàm số của numpy)

```

self.P = .....
.....
self.L = .....
.....

```

Câu 1.2 (1 điểm)

Xác định công thức tính giá trị **r** và **theta** trong hàm `__init__(self, n_points, n_classes, n_dimensions)`

```

r = .....
.....
theta = .....
.....

```

Bài 2: Sử dụng mạng nơ-ron nhân tạo để huấn luyện nhận biết đám mây điểm (8 điểm)

(**Lưu ý:** Các class được sử dụng cho mạng nơ-ron được định nghĩa tại **phụ lục**)

Trong bài này, ta sử dụng giải thuật ADAM (Adaptive Moment Optimizer) để tối ưu hóa các thông số của mạng nơ-ron. Cho class `Optimizer_Adam` (thuộc `Optimizer.py`) như định nghĩa bên dưới:

```

import numpy as np
class Optimizer_Adam:

    def __init__(self, learning_rate = 0.1, decay = 5e-3, epsilon = 1e-7,
        beta1 = 0.9, beta2 = 0.9):
        self.learning_rate = learning_rate

```



```

self.current_learning_rate = learning_rate
self.decay = decay
self.step = 0
self.epsilon = epsilon
self.beta1 = beta1
self.beta2 = beta2

# Cập nhật learning rate
def pre_update_params(self):
    if self.decay:
        self.current_learning_rate = self.learning_rate * (1 / (1 +
self.decay * self.step))

# Cập nhật các thông số mạng nơ-ron
def update_params(self, layer):

    if not hasattr(layer, 'weights_cache'):
        layer.weights_momentum = np.zeros_like(layer.weights)
        layer.weights_cache = np.zeros_like(layer.weights)
        layer.biases_momentum = np.zeros_like(layer.biases)
        layer.biases_cache = np.zeros_like(layer.biases)

    # Cập nhật giá trị động lượng (momentum)
    layer.weights_momentum = self.beta1 * layer.weights_momentum + (1 -
self.beta1) * layer.dweights
    layer.biases_momentum = self.beta1 * layer.biases_momentum + (1 -
self.beta1) * layer.dbiases

    # Hiệu chỉnh giá trị momentum
    weights_momentum_corrected = layer.weights_momentum / (1 -
self.beta1**(self.step + 1))
    biases_momentum_corrected = layer.biases_momentum / (1 -
self.beta1**(self.step + 1))

    # cập nhật các giá trị lưu trữ (cache)
    layer.weights_cache = self.beta2 * layer.weights_cache + (1-
self.beta2) * layer.dweights**2
    layer.biases_cache = self.beta2 * layer.biases_cache + (1-self.beta2)
* layer.dbiases**2

    # Cập nhật các giá trị momentum hiệu chỉnh với giá trị lưu trữ
    weights_cache_corrected = layer.weights_cache / (1 -
self.beta2**(self.step + 1))
    biases_cache_corrected = layer.biases_cache / (1 -
self.beta2**(self.step + 1))

    # Cập nhật trọng số (weights) và độ lệch (biases)
    layer.weights = ... # Câu 2.1
    layer.biases = ... # Câu 2.1

```

```
# Tăng biến đếm
def post_update_params(self):
    self.step += 1
```

Câu 2.1: Xác định công thức cho phép cập nhật lại dữ liệu của các trọng số (layer.weights) và các độ lệch (layer.biases) của mạng nơ-ron trong hàm update_params(self, layer) của class Optimizer_Adam. (1 điểm)

```
layer.weights = .....
.....
layer.biases = .....
.....
```

Chương trình python cho phép huấn luyện nhận biết đám mây điểm được viết như sau:

```
# Example of Training in loop: running for 10000 times

import numpy as np
import Activation as activation # type: ignore
import Dense as layer # type: ignore
import Optimizer as optimizer # type: ignore
import Points as point # type: ignore
import matplotlib.pyplot as plt

n_classes = 3
n_points = ... # Câu 2.2
Training_Data = ... # Câu 2.2

# Tạo một mạng thần kinh nhân tạo với 2 ngõ vào 2 lớp ẩn (32 nơ-ron / lớp) và
# 3 ngõ ra:
... # Câu 2.3

# Xác định hàm loss
loss_activation = ... # Câu 2.4

# Định nghĩa hàm tối ưu hóa của hệ thống
momentum = 0.4
# Adam parameters
learning_rate = 0.2
decay = 1e-5
epsilon = 1e-7
beta1 = 0.9
beta2 = 0.99

optimizer = ... # Câu 2.4

# Vòng lặp huấn luyện mạng nơ-ron
for epoch in range(10001):
```


Câu 2.2: Xác định các câu lệnh cho phép khởi tạo 3 đám mây điểm (3 lớp), mỗi đám mây có 300 điểm trong không gian 2D: (1 điểm)

Câu 2.3: Xác định các câu lệnh cho phép khởi tạo 1 mạng nơ-ron nhân tạo gồm 2 ngõ vào, 3 ngõ ra, và 2 lớp nơ-ron trung gian có Drop-out mỗi lớp có 32 nơ-ron.
Hàm activation của các lớp nơ-ron trung gian là ReLU.
Loại 5% nơ-ron ngẫu nhiên sau 2 lớp trung gian ở mỗi bước huấn luyện. (1.5 điểm)

[illegible]

Câu 2.7: Xác định các câu lệnh cho phép lan truyền ngược và cập nhật lại các thông số weights và biases của mạng nơ-ron (1.5 điểm)

This image shows a full page of primary-ruled notebook paper. It features a series of evenly spaced horizontal dotted lines for writing. A single vertical dotted line runs down the left side of the page, creating a margin. The paper is otherwise blank, with no handwriting or other markings.

Ghi chú: Cán bộ coi thi không được giải thích đề thi.

Chuẩn đầu ra của học phần (về kiến thức)	Nội dung kiểm tra
[CLO1]: Áp dụng kiến thức toán và khoa học để xây dựng mô hình tính toán cho các thuật toán	Bài 1, Bài 2 (Câu 2.1)
[CLO2]: Lập trình triển khai các thuật toán máy học trên máy tính và bo nhúng	Bài 2 (Câu 2.2, 2.3, 2.4, 2.5, 2.6, 2.7)

Chú ý: Cách thức bố trí các nội dung có thể tùy chỉnh cho phù hợp với đặc thù từng môn học, tuy nhiên cần đảm bảo tối thiểu các nội dung quy định trong biểu mẫu này.

Ngày 28 tháng 10 năm 2025

Trưởng bộ môn

(ký và ghi rõ họ tên)

Nguyễn Xuân Quang

Phụ lục

Các class cần có để khởi tạo và huấn luyện mạng nơ-ron

Class SpiralData (Thư viện Points.py)

```
import numpy as np
import matplotlib.pyplot as plt

class SpiralData:
    def __init__(self, n_points, n_classes, n_dimensions):

        self.N = n_points # Số lượng điểm ở mỗi class
        self.D = n_dimensions # Chiều không gian
        self.K = n_classes # Số lượng lớp (class)

        # Khởi tạo ma trận P chứa dữ liệu điểm và vec-tơ nhãn dán L
        self.P = ...
        self.L = ...

        for j in range(self.K):
            ix = range(self.N*j, self.N*(j+1))
            r = ...
            theta = ...
            self.P[ix] = np.c_[r*np.sin(theta), r*np.cos(theta)]
            self.L[ix] = j
```

Class Dense (Thư viện Dense.py)

```
import numpy as np
class Dense:
    def __init__(self, n_inputs, n_neurons):
        #init weights and biases
        self.weights = 0.01*np.random.randn(n_inputs, n_neurons)
        self.biases = np.zeros((1, n_neurons))

    def forward(self, inputs):
        #calculate outputs
        self.inputs = inputs
        self.output = np.dot(inputs, self.weights) + self.biases

    def backward(self, dvalues):
        # Gradients
        self.dweights = np.dot(self.inputs.T, dvalues)
        self.dbiases = np.sum(dvalues, axis=0, keepdims=True)
        self.dinputs = np.dot(dvalues, self.weights.T)
```


Class Loss (Loss.py)

```
import numpy as np
import math
```

```
# Loss Class
```

```
class Loss:
    def calculate(self,output,y):
        sample_losses = self.forward(output,y)
        data_loss = np.mean(sample_losses)
        return data_loss
```

```
# Cross Entropy Loss Class
```

```
class Loss_CategoricalCrossentropy(Loss):
```

```
    def forward(self, y_pred, y_true):
        samples = len(y_pred)
        y_pred_clipped = np.clip(y_pred,1e-7,1-1e-7)

        if len(y_true.shape) == 1:
            correct_confidence = y_pred_clipped[range(samples),y_true]
        elif len(y_true.shape) == 2:
            correct_confidence = np.sum(y_pred_clipped*y_true,axis = 1)

        negative_log_likelihoods = -np.log(correct_confidence)
        return negative_log_likelihoods

    def backward(self, dvalues, y_true):
        samples = len(dvalues)
        labels = len(dvalues[0])

        if len(y_true.shape) == 1:
            y_true = np.eye(labels)[y_true]

        self.dinputs = - y_true / dvalues
        self.dinputs = self.dinputs / samples
```

Class Activation (Activation.py)

```
import numpy as np
import Loss as loss
```

```
#Activation function ReLU
```

```
class Activation_ReLU:
```

```
    def forward(self,inputs):
        self.inputs = inputs
        self.output = np.maximum(0,inputs)

    def backward(self,dvalues):
        self.dinputs = dvalues.copy()
        self.dinputs[self.inputs <= 0] = 0
```



```

# Softmax Activation
class Activation_SoftMax():
    def forward(self,inputs):
        exp_values = np.exp(inputs - np.max(inputs, axis = 1,keepdims=True))
        self.exp_values = exp_values
        probabilities = exp_values / np.sum(exp_values, axis = 1, keepdims=True)
        self.output = probabilities

    def backward(self,dvalues):
        self.dinputs = np.empty_like(dvalues)
        for index, (single_output,single_dvalues) in enumerate(zip(self.output, dvalues)):
            single_output = single_output.reshape(-1,1)
            jacobian_matrix = np.diagflat(single_output) - np.dot(single_output,single_output.T)
            self.dinputs[index] = np.dot(jacobian_matrix,single_dvalues)

class Activation_Softmax_Loss_CategoricalCrossEntropy():

    def __init__(self):
        self.activation = Activation_SoftMax()
        self.loss = loss.Loss_CategoricalCrossentropy()

    def forward(self, inputs, y_true):
        self.activation.forward(inputs)
        self.output = self.activation.output
        return self.loss.calculate(self.output, y_true)

    def backward(self, dvalues, y_true):
        samples = len(dvalues)
        if len(y_true.shape) == 2:
            y_true = np.argmax(y_true, axis =1)

        self.dinputs = dvalues.copy()
        self.dinputs[range(samples), y_true] -= 1
        self.dinputs = self.dinputs / samples

```

Class Optimizer Adam: (Optimizer.py)

```

class Optimizer_Adam:

    def __init__(self, learning_rate = 0.1, decay = 5e-3, epsilon = 1e-7, beta1 =
0.9, beta2 = 0.9):
        self.learning_rate = learning_rate
        self.current_learning_rate = learning_rate
        self.decay = decay
        self.step = 0
        self.epsilon = epsilon
        self.beta1 = beta1
        self.beta2= beta2

```



```

def pre_update_params(self):
    if self.decay:
        self.current_learning_rate = self.learning_rate * (1 / (1 + self.decay
* self.step))

def update_params(self, layer):

    if not hasattr(layer, 'weights_cache'):

        layer.weights_momentum = np.zeros_like(layer.weights)
        layer.weights_cache = np.zeros_like(layer.weights)
        layer.biases_momentum = np.zeros_like(layer.biases)
        layer.biases_cache = np.zeros_like(layer.biases)

        # Update momentum with current gradient
        layer.weights_momentum = self.beta1 * layer.weights_momentum + (1 -
self.beta1) * layer.dweights
        layer.biases_momentum = self.beta1 * layer.biases_momentum + (1 -
self.beta1) * layer.dbiases

        # Correct the momentum. step must start with 1 here
        weights_momentum_corrected = layer.weights_momentum / (1 -
self.beta1**(self.step +1))
        biases_momentum_corrected = layer.biases_momentum / (1 -
self.beta1**(self.step +1))

        # update cache
        layer.weights_cache = self.beta2 * layer.weights_cache + (1-self.beta2) *
layer.dweights**2
        layer.biases_cache = self.beta2 * layer.biases_cache + (1-self.beta2) *
layer.dbiases**2

        # Obtain the corrected cache
        weights_cache_corrected = layer.weights_cache / (1 -
self.beta2**(self.step +1))
        biases_cache_corrected = layer.biases_cache / (1 - self.beta2**(self.step
+1))

        # Update weights and biases
        layer.weights += -self.current_learning_rate * weights_momentum_corrected
/ (np.sqrt(weights_cache_corrected) + self.epsilon)
        layer.biases += -self.current_learning_rate * biases_momentum_corrected /
(np.sqrt(biases_cache_corrected) + self.epsilon)

def post_update_params(self):
    self.step += 1

```